

程式語言的源頭與未來——Lisp 的前世今生

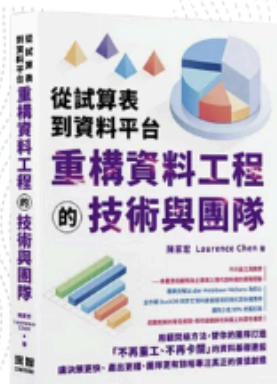
Laurence Chen / 陳家宏

- 台大電機系 (學士、碩士)
- Clojure, dbt-Taipei 線下活動主辦人
- IT 顧問、講者、作家
- 從試算表到資料平台－重構資料工程的技術與團隊

從試算表到資料平台 重構資料工程的技術與團隊

▲用顧問級方法，替團隊打造「不再重工、不再卡關」的資料基礎建設，讓決策更快、產出更穩、團隊更有餘裕專注真正的價值創造。

9/19 博客來/天瓏書局 正式發售



Outline

- 什麼是 Lisp ?
- Lisp 的起源
- Lisp 的現在
- Lisp 的未來

什麼是 Lisp ?

```
(+ 1 2)  
;; => 3
```

```
(+ (* 1 2)  
  (- 3 4))  
;; 求值的順序：從內往外  
;; => 1
```

什麼是 Lisp ? 函數 / if

```
;; define a function  
(defn add [x y]  
  (+ x y))
```

```
;; if branch  
(if cond  
  true-branch  
  false-branch)
```

什麼是 Lisp ? 迴圈/資料轉換

```
;; loop/recur - basic counter
(loop [i 0]
  (when (< i 5)
    (println i)
    (recur (inc i)))))
```

```
;; map - transform a collection
(map #(* % 2) [1 2 3 4 5])
```

```
;; reduce - fold a collection
(reduce + 0 [1 2 3 4 5])
```

起源 (Origin Story)

- John McCarthy 在研究 AI 時，創造了一種新的語言 Lisp (1960)
- 然後，有一連串的**意外**。

意外 1 - S-expression

- S-expression 本來只當作語言的內部表示法。
- 本來，John 打算要使用 M-expression 做為語言的外觀，但是，IBM 026 keypunch 沒有方括號。=> 最後，整個語言都是 **S-expression** 了。

```
M-expression => f[x;y]  
S-expression => (f x y)
```

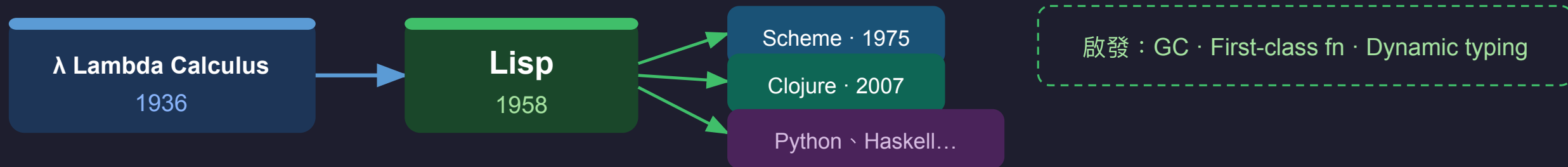
意外 2 - 理論 -> 工程

- 本來，只是用來寫 paper 而已。結果， Stephen R. Russell 和 Daniel J. Edwards 把 interpreter 實作出來。

意外 3 - 為 AI 而打造

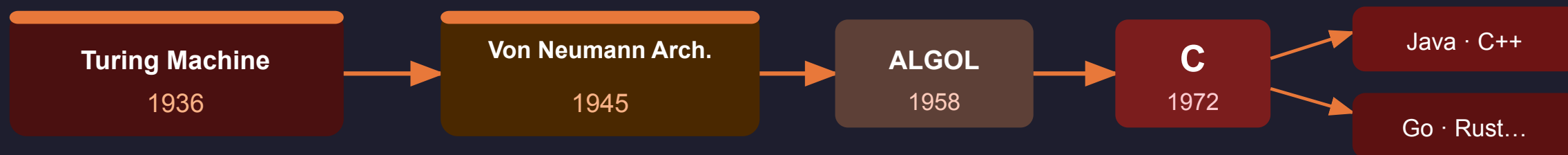
- 本來是為 AI 而打造。
 - AI 卻轉向了統計學習。
- 後來，Lisp 啟發了 programming languages 的設計與研究。
 - Garbage collection, First-class functions, Dynamic Types

函數式 · 動態型別



程式語言的起源

命令式 · 靜態型別



Metacircular Evaluator

- SICP 《Structure and Interpretation of Computer Programs》 電腦程式的構造和解釋
- 用 Lisp 寫一個 Lisp 直譯器 (100~150 LOC)
- 什麼？缺惰性求值？那改一下直譯器
- 如果你發現當前的語言限制了**表達能力**
 - 修改直譯器，創造一個新的語言層級來解決它。

現代的 Lisp

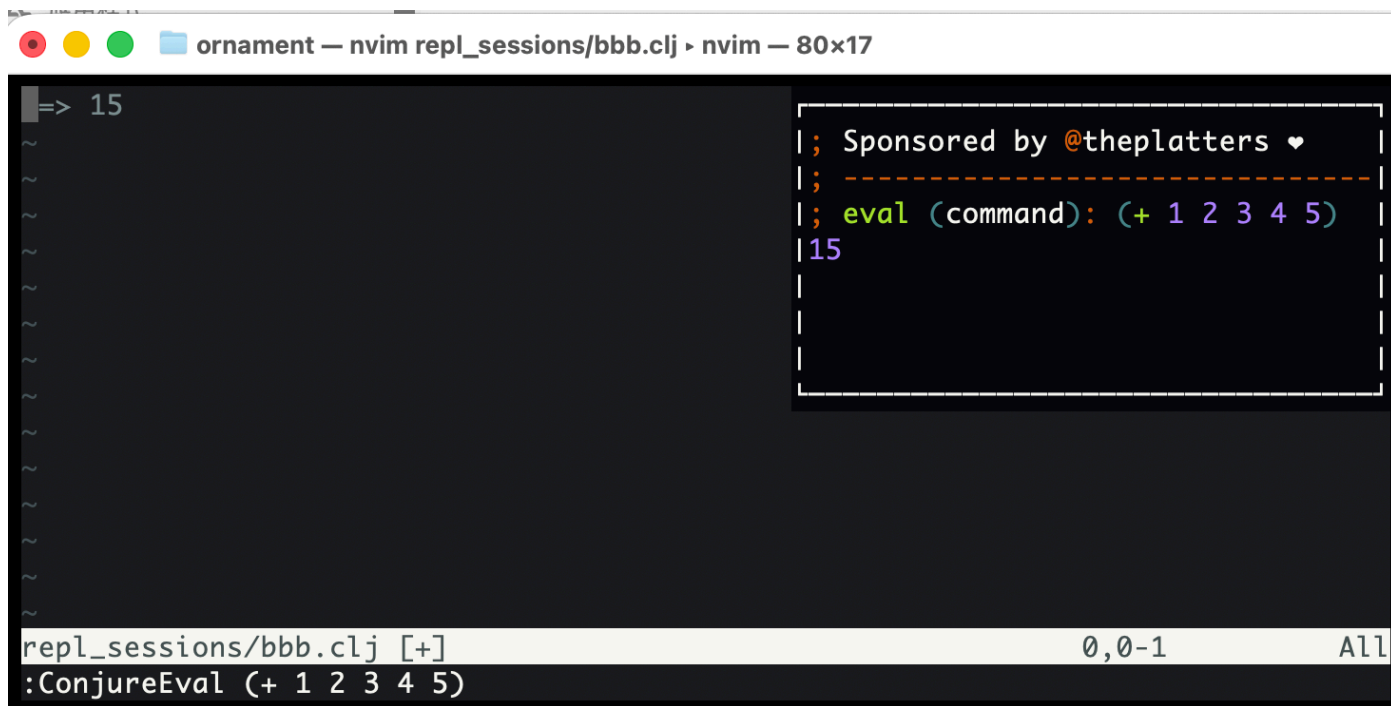
方言	應用	相對熱度
Clojure	on JVM	★★★★★
Common Lisp	傳統的 Lisp	★★★
Racket	學術、語言導向程式設計	★★★
Scheme	教學標準	★★
Fennel	on Lua VM	★

Lisp 在軟體開發中的四大特色

1. 互動式開發
2. S 表達式編輯
3. Macro
4. 資料導向編程

互動式開發 - 定義

- 觀念：開發軟體時，系統常駐啟動



The screenshot shows a terminal window titled "ornament — nvim repl_sessions/bbb.clj ▸ nvim — 80x17". The main area displays a REPL session with the prompt "=> 15" on the left and a dashed box on the right containing the following text:

```
; Sponsored by @theplatters ♥  
; -----  
; eval (command): (+ 1 2 3 4 5)  
15
```

The bottom status bar shows "repl_sessions/bbb.clj [+]" on the left, "0,0-1" in the center, and "All" on the right. Below the status bar, the command ":ConjureEval (+ 1 2 3 4 5)" is visible.

互動式開發 - 應用

- 不寫測試的 TDD (test-driven development)
- 即時求值
- 內聯檢查 (inline inspection)
- 對函式庫除錯

```
(defn reverse* [xs]
  (reduce (fn [coll x]
            (cons x coll)) '() xs))

(comment
  ;; 開發到一半，就來這邊 "eval" 一下。
  (reverse* [])
  (reverse* [1])
  (reverse* [1 2 3 4 5]))
```

```
| ; Sponsored by @jackkinsey ♥
| ; -----
| ; eval (current-form): (defn reverse* [xs] (reduce (f
| #'user/reverse*
| ; -----
| ; eval (current-form): (reverse* [])
| ()
| ; -----
| ; eval (current-form): (reverse* [1])
| (1)
| ; -----
| ; eval (current-form): (reverse* [1 2 3 4 5])
| (5 4 3 2 1)
```

互動式開發 - 內聯檢查

```
;; 想觀察區域變數 z 的值?  
(defn wrong-add [x y]  
  (let [z (+ x 2)]  
    (+ z y)))
```

```
(wrong-add 1 2)  
;; => 5
```

```
(defn wrong-add [x y]  
  (let [z (+ x 2)]  
    (def z z) ;; 用 def 捕捉 z  
    (+ z y)))
```

```
;; 再次執行  
(wrong-add 1 2)
```

```
;; 在 REPL 直接求值 z  
z ;; => 3
```

互動式開發 - 對函式庫除錯

- 懷疑某個函式庫的函數 `pp` 有問題？
- 做法：
 - i. 跳轉定義，進入函式庫原始碼
 - ii. 直接修改 `pp` 函數，**不存檔**，對 buffer 求值
 - iii. 直譯器裡的定義被替換，立即可以驗證假設
- `buffer` $\neq$ 硬碟上的檔案，關掉就還原

S 表達式編輯

- 觀念：語法樹顯式呈現
- 應用：IDE 提供強大的輔助編輯能力

S 表達式編輯 - formatting

- 不良的 formatting

```
(defn factorial [x]
  (if (< x 2)
      1
      (* x (factorial (- x 1)
                      )
          )
    )
)
```

S 表達式編輯 - formatting

- 正確的 formatting
- 括弧成對問題
 - 彩虹括弧
 - 括弧自動配對

```
(fn factorial [x]  
  (if (< x 2)  
      1  
      (* x (factorial (- x 1)))))
```

~

S 表達式編輯 - IDE 輔助編輯 1

- 括弧編輯
 - 圍繞括弧、刪去括弧

S 表達式編輯 - IDE 輔助編輯 2

- 在語法樹裡穿梭編輯
 - 游標快速移動
 - 移動元素
 - 刪除元素

S 表達式編輯 - IDE 輔助編輯 3

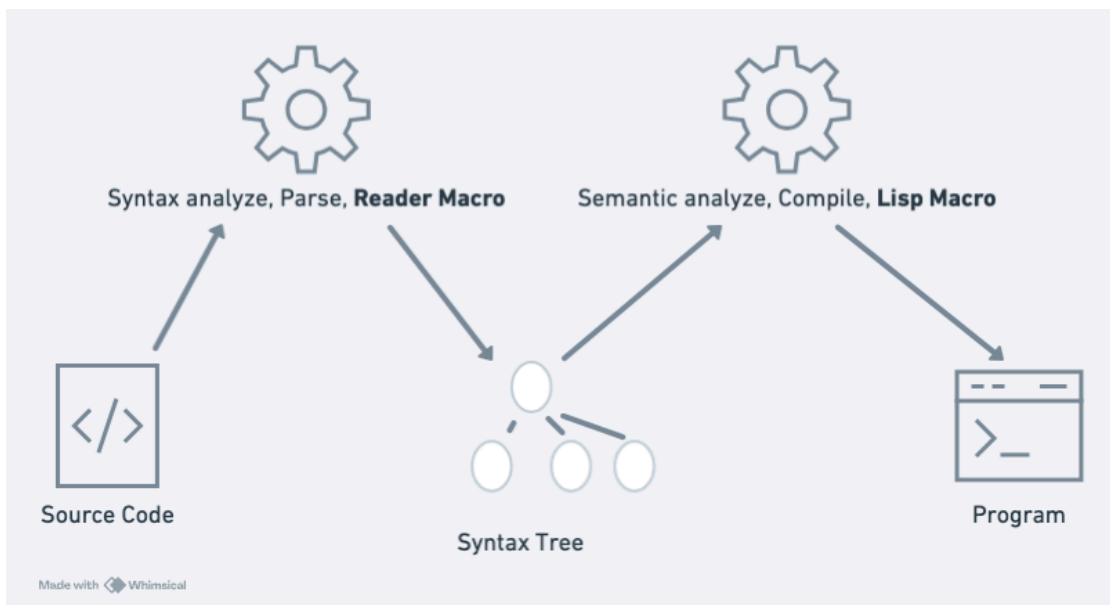
- 交換 true/false branch

```
;; before
(if is-admin
  (show-dashboard)
  (show-login))

;; 游標移到 (show-dashboard)，下一個 >e 指令
;; after
(if is-admin
  (show-login)
  (show-dashboard))
```

Macro - 原理

- 暴露出來的 2 個 phase 與 2 種 plugin
- Reader macro :
 - `'x -> (quote x)`
- Lisp macro :
 - `(when condition body) -> (if condition body nil)`



Macro - 謹慎使用

- 觀念：Macro 可以操作程式碼結構
- 應用：
 - 創造新的語法
 - 上下文管理
- 何時使用？
 - 當它能 pull the complexity downwards 時——讓呼叫端簡化，而代價是將複雜性藏入 Macro 裡。

Macro - threading macro (控制語法)

```
(-> 5 (+ 1) (* 10) math.abs)
```

to

```
(math.abs (* 10 (+ 1 5)))
```

Macro - threading macro 的實作

```
(defmacro ->
  [x & forms]
  (loop [x x, forms forms]
    (if forms
      (let [form (first forms)]
        threaded (if (seq? form)
                    (with-meta `(~(first form) ~x ~@(next form)) (meta form))
                    (list form x)))
      (recur threaded (next forms)))
    x)))
```

Macro - 上下文管理

```
(with-open [r (clojure.java.io/input-stream "myfile.txt")]  
  (loop [c (.read r)]  
    (if (not= c -1)  
      (do  
        (print (char c))  
        (recur (.read r)))))))
```

Macro - Python 的 Context Manager

```
# 沒有使用 with 的寫法
f = open('test.txt', 'r')
try:
    content = f.read()
    # 即使這裡發生錯誤，finally 區塊的 f.close() 依然會被執行。
    # 這種寫法比較繁瑣，需要手動處理資源關閉。
    print(content)
finally:
    f.close()
```

Macro - Python 的 Context Manager

```
# 使用 with 的寫法
with open('test.txt', 'r') as f:
    content = f.read()
# 這裡的程式碼如果發生錯誤，`with` 會確保 `f` 物件的
# `__exit__` 方法被自動呼叫，從而正確關閉檔案。
# 這種寫法更簡潔、安全。
print(content)
```

資料導向編程 - 定義

- 觀念：Context 賦予資料語義，於是邏輯可以透過資料結構來表達。
- 應用：DSL

資料導向編程 - 樹的表達



(祖父 (父親 (我)
 (妹妹))
 (伯父))

資料導向編程 - 向量作為路徑邏輯

```
(def data {:a {:b {:c 100}}})
```

```
(get-in data [:a :b :c])  
;; => 100
```

;; [:a :b :c] 不只是向量，它描述了「如何取值」的邏輯

資料導向編程 - 集合作為篩選邏輯

```
(def data-seq [:a :c :b :d])
```

```
(filter #{:a :b} data-seq)  
;; => (:a :b)
```

;; #{:a :b} 不只是集合，它就是篩選的規則

Code is Data vs Data is Code

- **Code is Data** (傳統 Lisp)
 - 程式碼是 S 表達式，而 S 表達式是一種資料結構
 - Macro 可以操作程式碼 => 用程式寫程式
- **Data is Code** (Clojure 的延伸)
 - 資料結構被上下文賦予語意，成為了 DSL
 - `[:a :b :c]` 是路徑，`#{:a :b}` 是規則
 - 資料結構本身就能承載邏輯
- 此資料結構非彼資料結構

Lisp 的未來

- 一般表示法收斂到了樹狀結構：XML, HTML, JSON
- Clojure/Lisp 遷移到各個平台
- IDE 插件
- Codemod
- Transpile

表示法收斂到樹狀結構

```
<!-- HTML -->  
<div>  
  <p>Hello</p>  
</div>
```

```
;; Hiccup (Clojure) – 直接用資料結構表達 HTML  
[:div  
  [:p "Hello"]]
```

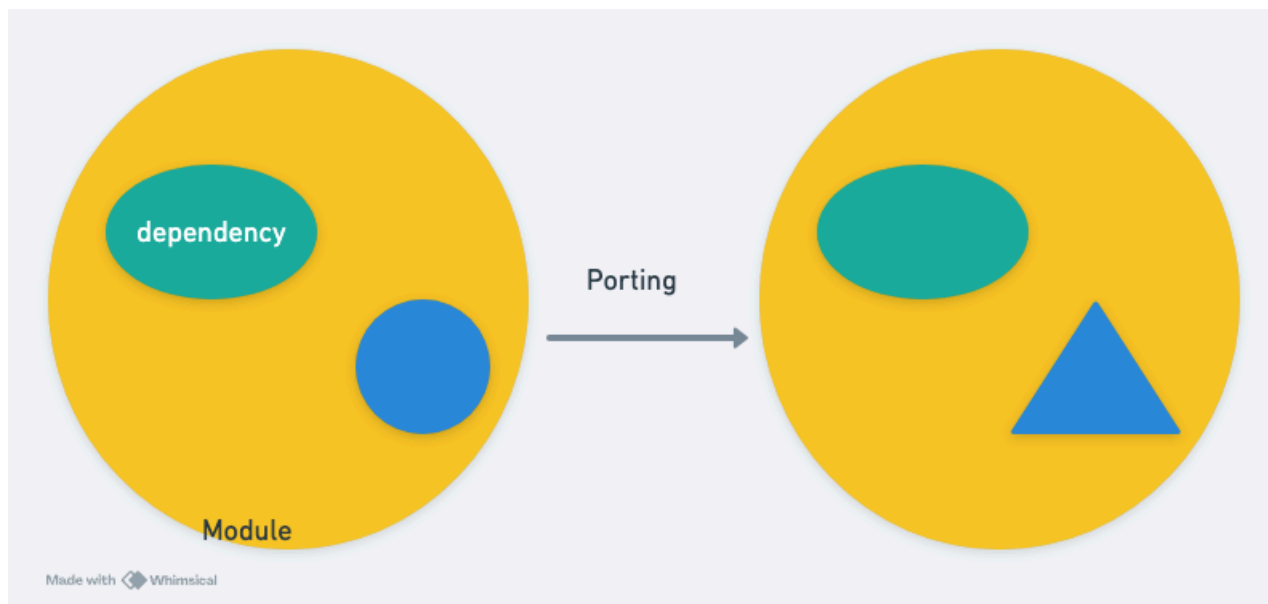
- HTML、XML、JSON 本質上都是樹狀資料
- Lisp 的 S 表達式從 1958 年就這樣做了

可以使用 Clojure 方言的平台

- JVM -> Clojure
- JavaScript -> ClojureScript
- Dot Net -> ClojureCLR
- Shell -> Babashka
- Lua -> Fennel

Lisp 怎麼遷移？

- Clojure => 將 Lisp 編譯成為 JVM bytecode
- Hylang => 將 Lisp 編譯成為 Python AST
- ClojureScript => 將 Lisp 編譯成為 JavaScript
- Fennel => 將 Lisp 編譯成為 Lua



開發 IDE 插件 - Tree-sitter

- `:InspectTree` (檢視語法樹)

```
conjure — nvim fnl/conjure/client/fennel/def-str-util.fnl ▸ lua — 133x33

(program ; [0, 0] - [164, 0]
  (local_form ; [0, 0] - [0, 51]
    call: (symbol) ; [0, 1] - [0, 6]
    (binding_pair ; [0, 7] - [0, 50]
      lhs: (table_binding ; [0, 7] - [0, 19]
        item: (table_binding_pair ; [0, 8] - [0, 18]
          key: (symbol_binding) ; [0, 8] - [0, 9]
          value: (symbol_binding)) ; [0, 10] - [0, 18]
        rhs: (list ; [0, 20] - [0, 50]
          call: (symbol) ; [0, 21] - [0, 28]
          item: (string ; [0, 29] - [0, 49]
            content: (string_content))) ; [0, 30] - [0, 49]
      (local_form ; [1, 0] - [1, 42]
        call: (symbol) ; [1, 1] - [1, 6]
        (binding_pair ; [1, 7] - [1, 41]
          lhs: (symbol_binding) ; [1, 7] - [1, 11]
          rhs: (list ; [1, 12] - [1, 41]
            call: (symbol) ; [1, 13] - [1, 21]
            item: (string ; [1, 22] - [1, 40]
              content: (string_content)))) ; [1, 23] - [1, 40]
      (local_form ; [2, 0] - [2, 50]
        call: (symbol) ; [2, 1] - [2, 6]
        (binding_pair ; [2, 7] - [2, 49]
          lhs: (symbol_binding) ; [2, 7] - [2, 17]
          rhs: (list ; [2, 18] - [2, 49]
            call: (symbol) ; [2, 19] - [2, 27]
            item: (string ; [2, 28] - [2, 48]
              content: (string_content)))) ; [2, 29] - [2, 48]
      (local_form ; [3, 0] - [3, 53]
        call: (symbol) ; [3, 1] - [3, 6]
        (binding_pair ; [3, 7] - [3, 52]
          lhs: (symbol_binding) ; [3, 7] - [3, 17]
          rhs: (list ; [3, 18] - [3, 52]
            call: (symbol) ; [3, 19] - [3, 27]
            item: (string ; [3, 28] - [3, 48]
              content: (string_content)))) ; [3, 29] - [3, 48]
      (local_form ; [4, 0] - [4, 50]
        call: (symbol) ; [4, 1] - [4, 6]
        (binding_pair ; [4, 7] - [4, 49]
          lhs: (symbol_binding) ; [4, 7] - [4, 17]
          rhs: (list ; [4, 18] - [4, 49]
            call: (symbol) ; [4, 19] - [4, 27]
            item: (string ; [4, 28] - [4, 48]
              content: (string_content)))) ; [4, 29] - [4, 48]
      (local_form ; [5, 0] - [5, 50]
        call: (symbol) ; [5, 1] - [5, 6]
        (binding_pair ; [5, 7] - [5, 49]
          lhs: (symbol_binding) ; [5, 7] - [5, 17]
          rhs: (list ; [5, 18] - [5, 49]
            call: (symbol) ; [5, 19] - [5, 27]
            item: (string ; [5, 28] - [5, 48]
              content: (string_content)))) ; [5, 29] - [5, 48]
      (local_form ; [6, 0] - [6, 50]
        call: (symbol) ; [6, 1] - [6, 6]
        (binding_pair ; [6, 7] - [6, 49]
          lhs: (symbol_binding) ; [6, 7] - [6, 17]
          rhs: (list ; [6, 18] - [6, 49]
            call: (symbol) ; [6, 19] - [6, 27]
            item: (string ; [6, 28] - [6, 48]
              content: (string_content)))) ; [6, 29] - [6, 48]
      (local_form ; [7, 0] - [7, 50]
        call: (symbol) ; [7, 1] - [7, 6]
        (binding_pair ; [7, 7] - [7, 49]
          lhs: (symbol_binding) ; [7, 7] - [7, 17]
          rhs: (list ; [7, 18] - [7, 49]
            call: (symbol) ; [7, 19] - [7, 27]
            item: (string ; [7, 28] - [7, 48]
              content: (string_content)))) ; [7, 29] - [7, 48]
      (local_form ; [8, 0] - [8, 50]
        call: (symbol) ; [8, 1] - [8, 6]
        (binding_pair ; [8, 7] - [8, 49]
          lhs: (symbol_binding) ; [8, 7] - [8, 17]
          rhs: (list ; [8, 18] - [8, 49]
            call: (symbol) ; [8, 19] - [8, 27]
            item: (string ; [8, 28] - [8, 48]
              content: (string_content)))) ; [8, 29] - [8, 48]
      (local_form ; [9, 0] - [9, 50]
        call: (symbol) ; [9, 1] - [9, 6]
        (binding_pair ; [9, 7] - [9, 49]
          lhs: (symbol_binding) ; [9, 7] - [9, 17]
          rhs: (list ; [9, 18] - [9, 49]
            call: (symbol) ; [9, 19] - [9, 27]
            item: (string ; [9, 28] - [9, 48]
              content: (string_content)))) ; [9, 29] - [9, 48]
      (local_form ; [10, 0] - [10, 50]
        call: (symbol) ; [10, 1] - [10, 6]
        (binding_pair ; [10, 7] - [10, 49]
          lhs: (symbol_binding) ; [10, 7] - [10, 17]
          rhs: (list ; [10, 18] - [10, 49]
            call: (symbol) ; [10, 19] - [10, 27]
            item: (string ; [10, 28] - [10, 48]
              content: (string_content)))) ; [10, 29] - [10, 48]
      (local_form ; [11, 0] - [11, 50]
        call: (symbol) ; [11, 1] - [11, 6]
        (binding_pair ; [11, 7] - [11, 49]
          lhs: (symbol_binding) ; [11, 7] - [11, 17]
          rhs: (list ; [11, 18] - [11, 49]
            call: (symbol) ; [11, 19] - [11, 27]
            item: (string ; [11, 28] - [11, 48]
              content: (string_content)))) ; [11, 29] - [11, 48]
      (local_form ; [12, 0] - [12, 50]
        call: (symbol) ; [12, 1] - [12, 6]
        (binding_pair ; [12, 7] - [12, 49]
          lhs: (symbol_binding) ; [12, 7] - [12, 17]
          rhs: (list ; [12, 18] - [12, 49]
            call: (symbol) ; [12, 19] - [12, 27]
            item: (string ; [12, 28] - [12, 48]
              content: (string_content)))) ; [12, 29] - [12, 48]
      (local_form ; [13, 0] - [13, 50]
        call: (symbol) ; [13, 1] - [13, 6]
        (binding_pair ; [13, 7] - [13, 49]
          lhs: (symbol_binding) ; [13, 7] - [13, 17]
          rhs: (list ; [13, 18] - [13, 49]
            call: (symbol) ; [13, 19] - [13, 27]
            item: (string ; [13, 28] - [13, 48]
              content: (string_content)))) ; [13, 29] - [13, 48]
      (local_form ; [14, 0] - [14, 50]
        call: (symbol) ; [14, 1] - [14, 6]
        (binding_pair ; [14, 7] - [14, 49]
          lhs: (symbol_binding) ; [14, 7] - [14, 17]
          rhs: (list ; [14, 18] - [14, 49]
            call: (symbol) ; [14, 19] - [14, 27]
            item: (string ; [14, 28] - [14, 48]
              content: (string_content)))) ; [14, 29] - [14, 48]
      (local_form ; [15, 0] - [15, 50]
        call: (symbol) ; [15, 1] - [15, 6]
        (binding_pair ; [15, 7] - [15, 49]
          lhs: (symbol_binding) ; [15, 7] - [15, 17]
          rhs: (list ; [15, 18] - [15, 49]
            call: (symbol) ; [15, 19] - [15, 27]
            item: (string ; [15, 28] - [15, 48]
              content: (string_content)))) ; [15, 29] - [15, 48]
      (local_form ; [16, 0] - [16, 50]
        call: (symbol) ; [16, 1] - [16, 6]
        (binding_pair ; [16, 7] - [16, 49]
          lhs: (symbol_binding) ; [16, 7] - [16, 17]
          rhs: (list ; [16, 18] - [16, 49]
            call: (symbol) ; [16, 19] - [16, 27]
            item: (string ; [16, 28] - [16, 48]
              content: (string_content)))) ; [16, 29] - [16, 48]
      (local_form ; [17, 0] - [17, 50]
        call: (symbol) ; [17, 1] - [17, 6]
        (binding_pair ; [17, 7] - [17, 49]
          lhs: (symbol_binding) ; [17, 7] - [17, 17]
          rhs: (list ; [17, 18] - [17, 49]
            call: (symbol) ; [17, 19] - [17, 27]
            item: (string ; [17, 28] - [17, 48]
              content: (string_content)))) ; [17, 29] - [17, 48]
      (local_form ; [18, 0] - [18, 50]
        call: (symbol) ; [18, 1] - [18, 6]
        (binding_pair ; [18, 7] - [18, 49]
          lhs: (symbol_binding) ; [18, 7] - [18, 17]
          rhs: (list ; [18, 18] - [18, 49]
            call: (symbol) ; [18, 19] - [18, 27]
            item: (string ; [18, 28] - [18, 48]
              content: (string_content)))) ; [18, 29] - [18, 48]
      (local_form ; [19, 0] - [19, 50]
        call: (symbol) ; [19, 1] - [19, 6]
        (binding_pair ; [19, 7] - [19, 49]
          lhs: (symbol_binding) ; [19, 7] - [19, 17]
          rhs: (list ; [19, 18] - [19, 49]
            call: (symbol) ; [19, 19] - [19, 27]
            item: (string ; [19, 28] - [19, 48]
              content: (string_content)))) ; [19, 29] - [19, 48]
      (local_form ; [20, 0] - [20, 50]
        call: (symbol) ; [20, 1] - [20, 6]
        (binding_pair ; [20, 7] - [20, 49]
          lhs: (symbol_binding) ; [20, 7] - [20, 17]
          rhs: (list ; [20, 18] - [20, 49]
            call: (symbol) ; [20, 19] - [20, 27]
            item: (string ; [20, 28] - [20, 48]
              content: (string_content)))) ; [20, 29] - [20, 48]
      (local_form ; [21, 0] - [21, 50]
        call: (symbol) ; [21, 1] - [21, 6]
        (binding_pair ; [21, 7] - [21, 49]
          lhs: (symbol_binding) ; [21, 7] - [21, 17]
          rhs: (list ; [21
```

開發 IDE 插件 - Tree-sitter Query

- :EditQuery (寫 S 表達式來查詢語法樹)

- InspectTree

- EditQuery

```
(fn search-in-ext-buffer [code-text last-row bufnr] fn
  "Search defs inside one buffer"
  (let [curr-targets (search-ext-targets def-query (get-current
    results (core.filter (fn [node-t]
      (= code-text node-t.content))
      curr-targets))]
    results)))

(fn jump-to-range [range] fn
  (vim.api.nvim_win_set_cursor 0 range.start))

(fn rest-str [s] fn
  "equal to clojure's (str (rest s))"
  (string.sub s 2 -1))

(fn resolve-lua-module-path [modname] fn
  "Try to resolve a lua module via package.path"
  (package.searchpath (.. "lua." modname) package.path))

(fn resolve-fnl-module-path [modname] fn
  "Try to resolve a fnl module via fennel.path"
  (package.searchpath modname (. (config.default) :fennel-path)))
```

```
rhs: (list ; [65, 18] - [65, 41]
      call: (multi_symbol ; [65, 19] - [65, 35]
             base: (symbol_fragment) ; [65, 19] - [65, 29]
             member: (symbol_fragment)) ; [65, 30] - [65, 35]
             item: (symbol))) ; [65, 36] - [65, 40]
      (binding_pair ; [66, 12] - [66, 52])

<or fnl/conjure/client/fennel/def-str-util.fnl 311,1          35%
;; Write queries here (see $VIMRUNTIME/queries/ for examples).
;; Move cursor to a capture ("@foo") to highlight matches in the
;; Completion for grammar nodes is available (:help compl-omni)

(fn_form
 name: (symbol) @fn)
~
~
~
~
~
~
~
~
```

Codemod

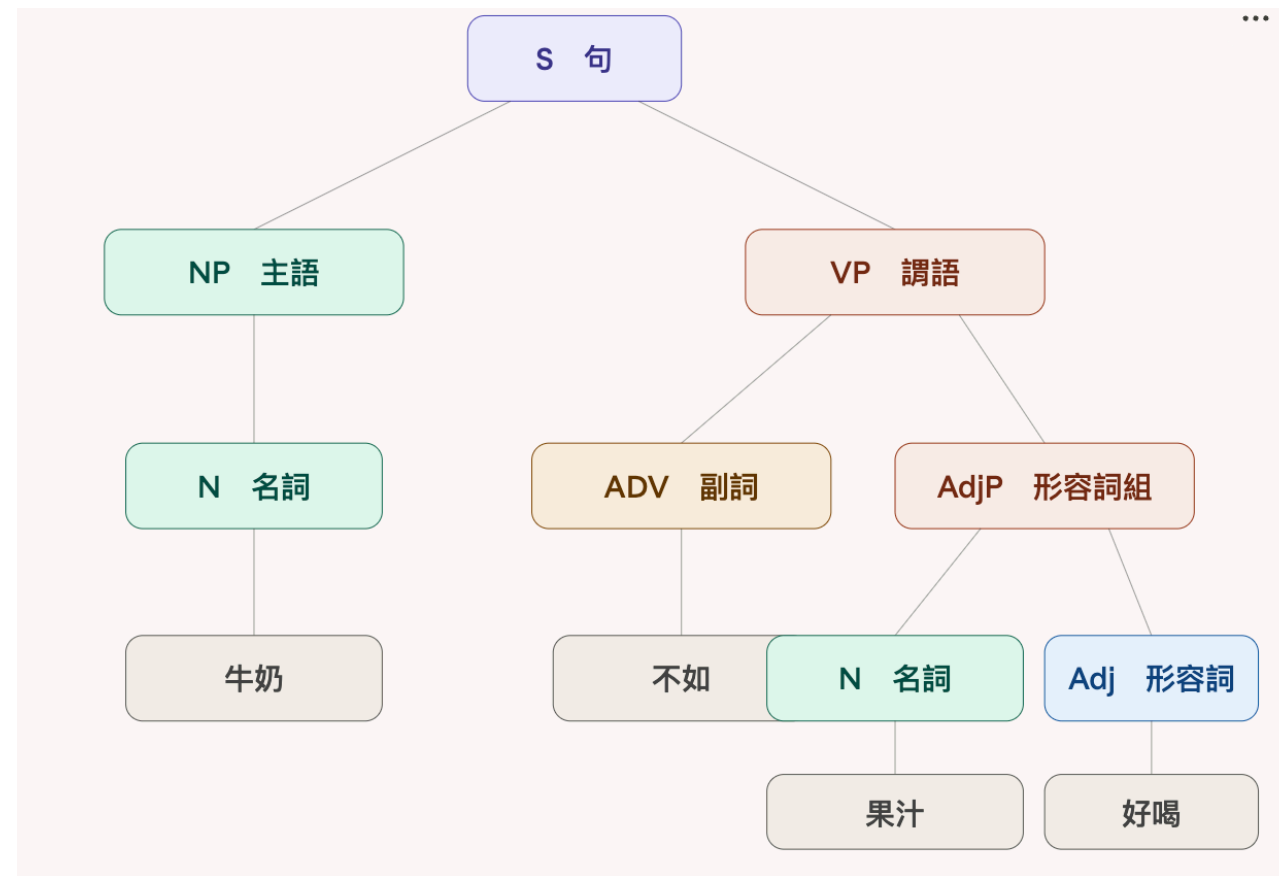
- Codemod (code modification) 是用程式來批次修改程式碼的技術。核心概念是：與其人工一個個改，不如寫一個「轉換腳本」，讓它自動處理整個 codebase。
- Why not find and replace?

Codemod - problem

```
# 你想把 config.get() 改成 config.fetch()  
# 但字串取代會連這個也動到：  
print("please use config.get() for settings")  
config_getter = config.get # 這語意不同
```

Codemod - 通俗

- 牛奶不如果汁好喝



Codemod - solution

- AST (abstract syntax tree)
- 操作 AST 節點，只修改語意上正確的位置
- 字串相似不代表語意相同，AST 不會搞混

Codemod - tool

- JavaScript
 - jscodeshift (Meta 出品，最主流)
- Python
 - libcst — lossless，保留空白與註解，Meta 內部主力工具
- 跨語言
 - ast-grep — 支援多語言，用類似程式碼的 pattern 來描述要找什麼

Transpile - sqlglot

- 各家的雲端的 Data Warehouse 都是 closed source 的
- 資料轉換是用 SQL 寫的，想要做 unit test
- 用 sqlglot 將 BigQuery SQL 轉成 DuckDB SQL 來測。(解析 AST)

結論 - insights

- Lisp 的誕生：偶然的創新
- Lisp 的失敗：AI 敗給了統計學習
- Lisp 的啟發：互動式開發、Lisp Macro、DSL
- Lisp 為何如此管用？
 - 程式碼是帶有**隱性結構**的資料
 - 語法樹讓我們像操作資料庫一樣，精準地理解、查詢、操作程式碼

結論 - call to action

- 選一種 Lisp 來學？
- Clojure
 - HoneySQL
 - Polylith
 - Rama
 - Extract reusable code into **library**